

# STARTUPOPS AI

## Complete Project Specification & Antigravity Implementation Blueprint

Version 1.0 | 28 September 2026 | Production-grade MVP to advanced platform

Build an AI operating layer for startups that connects to live work systems, investigates cross-system problems, prioritizes attention, proposes actions, and executes approved actions. This is not a generic chatbot, upload-based RAG app, or simulator.

### 1. Product Vision

- StartupOps AI is a persistent internal operations agent for startups. A user talks to one agent while it delegates work to specialized integration tools.
- Core loop: Observe → Understand → Correlate → Prioritize → Propose → Approve → Execute → Verify → Learn.
- Primary users: founders, engineering leads, customer/support leads, sales/revenue leads and operations teams.

### 2. Non-Goals

- Do not build a generic ChatGPT clone or upload-first RAG application.
- Do not require Ollama. Use Gemini through Google AI Studio API.
- Never automatically send external messages, delete data, modify production systems, or perform other consequential actions without required approval.
- Never present an unsupported causal claim as fact.

### 3. User Experience

- First launch: create workspace → connect integrations via OAuth → choose role/preferences → test connections → run initial health scan.
- Main screens: Command Center, Today, Investigations, Signals, Approval Center, Integrations, Audit/Activity, Settings.
- Example commands: What needs my attention today? Investigate checkout complaints. Show stale engineering work. Prepare follow-ups. Why did support volume increase?

### 4. Architecture

- Frontend: Next.js + TypeScript + Tailwind CSS.
- Backend: Python + FastAPI.
- Agent orchestration: LangGraph with explicit state and tool permissions.
- LLM: Gemini via Google AI Studio API. Key stays server-side and is supplied through environment/secret management.
- Data: PostgreSQL; pgvector where semantic search is needed.
- Jobs: Redis plus a background worker system.
- Integrations: GitHub, Gmail, Slack first; Calendar, CRM, Jira/Linear, support, analytics and payments later.
- Playwright only for allowlisted domains when an API is genuinely unavailable.

### 5. Agent Architecture

- Supervisor Agent: understands intent, selects workflow and enforces policy.
- Planner: decomposes requests into typed steps.
- GitHub Agent: issues, PRs, commits, reviews, CI and repository activity.
- Email Agent: search/read, draft, and approved sending.
- Slack Agent: search/read, draft, and approved posting.
- Investigation Agent: gathers evidence across systems.
- Signal Agent: anomaly detection, clustering and cross-system patterns.

- Action Agent: converts findings into explicit actions.
- Verifier Agent: confirms executed actions actually succeeded.
- Sub-agents must never bypass the central permission layer.

## 6. Core Cross-System Workflow

- Normalize events into a common model: actor, object, action, timestamp, source, workspace and metadata.
- Example: support complaints → customer/topic cluster → recent release → changed files → PR → owner → Slack discussion.
- Represent outputs as evidence, association or hypothesis. Do not automatically call an association a root cause.
- Every finding must link to the evidence used to produce it.

## 7. Agent State

- Persist typed state containing workspace\_id, user\_id, request\_id, goal, plan, evidence\_refs, tool\_calls, findings, hypotheses, risk\_level, proposed\_actions, approvals, execution\_results, verification\_results, errors and final\_response.
- Persist checkpoints for resumability and use idempotency keys for writes.

## 8. Initial Integrations

- GitHub OAuth: repositories, issues, PRs, commits, reviews and workflow/check data.
- Gmail OAuth: search/read and create drafts; sending is an approved write.
- Slack OAuth: permitted search/read; posting is an approved write.
- Encrypt provider tokens at rest. Store scopes and external account IDs.

## 9. Database

- Core tables: users, workspaces, workspace\_members, roles, integration\_accounts, integration\_scopes, encrypted\_credentials, conversations, messages, agent\_runs, agent\_steps, evidence\_items, signals, investigations, findings, hypotheses, action\_plans, actions, approvals, execution\_results, verification\_results, audit\_logs, sync Cursors, notifications, model\_usage.
- Tenant-owned rows must carry workspace\_id or belong to a workspace-owned parent. Add indexes, timestamps, provider IDs and unique constraints.
- Key entities: workspaces; workspace\_members; integration\_accounts; agent\_runs; evidence\_items; signals; action\_plans; actions; approvals; audit\_logs.

## 10. Signal Detection & ML

- Start with deterministic rules, rolling baselines, robust statistics, embeddings and LLM reasoning.
- Signals: support-volume spikes, repeated complaints, aging PRs, CI failures, unanswered sales threads, post-release complaint spikes and operational overload.
- Later add XGBoost for prioritization, PyTorch transformers for event-sequence modeling and SHAP for explainability.
- ML is advisory and traceable to features/evidence.

## 11. Action & Approval System

- Lifecycle: proposed → awaiting\_approval → approved/rejected → executing → succeeded/failed → verified.
- Approval UI shows exact target, parameters, intended effect, permissions, risk and evidence.
- High-risk actions require explicit confirmation. Support single and batch approval.
- LLM output alone can never constitute approval.

## 12. Security

- OAuth 2.0 with least privilege. Encrypt tokens and sensitive data.
- Never put Gemini keys or provider secrets in frontend code, prompts, logs or Git.

- Enforce workspace authorization on every backend request.
- Validate all tool inputs with Pydantic/JSON Schema.
- Protect against SSRF and arbitrary browser navigation; use domain/action allowlists.
- Audit authentication, tool calls, approvals, writes and failures.
- Treat all email, Slack, GitHub and web content as untrusted data; defend against prompt injection.

## 13. API

- POST /api/v1/workspaces
- GET /api/v1/integrations
- POST /api/v1/integrations/{provider}/connect
- POST /api/v1/agent/runs
- GET /api/v1/agent/runs/{id}
- GET /api/v1/signals
- POST /api/v1/investigations
- GET /api/v1/action-plans/{id}
- POST /api/v1/actions/{id}/approve
- POST /api/v1/actions/{id}/reject
- POST /api/v1/actions/{id}/execute
- GET /api/v1/audit
- POST /api/v1/sync/{provider}

## 14. Frontend Screens

- Login/workspace creation; Command Center; Today; Investigation workspace; Signals; Approval Center; Integrations; Audit/Activity; Settings.
- Investigation view must show timeline, evidence, hypotheses, confidence, proposed actions and execution status.

## 15. Repository Structure

- frontend/ — Next.js application
- backend/ — FastAPI application
- backend/app/agents/ — supervisor, planner, investigator, verifier
- backend/app/integrations/ — github, gmail, slack
- backend/app/tools/ — typed read/write tools
- backend/app/policies/ — authorization, risk and approval policies
- backend/app/models/ — SQLAlchemy/Pydantic models
- backend/app/workers/ — background jobs
- backend/app/services/ — sync, evidence, signals, actions, audit
- backend/tests/ — unit/integration/security tests
- infra/ — Docker/deployment
- docs/ — architecture, API, security, runbooks

## 16. Development Phases

- Phase 1 Foundation: repo, UI shell, FastAPI, PostgreSQL, authentication, tenancy, secrets and audit.
- Phase 2 Integrations: GitHub/Gmail/Slack OAuth, read-only sync and normalized evidence.
- Phase 3 Agent: Gemini, LangGraph supervisor/planner and tool registry.

- Phase 4 Investigation: cross-system search, timeline, clustering and signal detection.
- Phase 5 Actions: risk engine, approval center, execution and verification.
- Phase 6 ML: anomaly detection, prioritization and transformer research.
- Phase 7 Hardening: security, tenant isolation, prompt-injection tests, rate limits, observability and cost controls.
- Phase 8 Deployment: containers, managed PostgreSQL/Redis, secrets, OAuth callbacks, backups and monitoring.

## 17. Environment Variables

- GOOGLE\_AI\_STUDIO\_API\_KEY, APP\_SECRET\_KEY, DATABASE\_URL, REDIS\_URL, ENCRYPTION\_KEY, GITHUB\_CLIENT\_ID, GITHUB\_CLIENT\_SECRET, GOOGLE\_CLIENT\_ID, GOOGLE\_CLIENT\_SECRET, SLACK\_CLIENT\_ID, SLACK\_CLIENT\_SECRET, FRONTEND\_URL, BACKEND\_URL, ENVIRONMENT, LOG\_LEVEL.
- Create .env.example only. Never commit real credentials.

## 18. Testing & Evaluation

- Unit tests: policy engine, schemas, signal rules, correlation and action state machine.
- Integration tests: OAuth, provider adapters, sync cursors and database transactions.
- Agent tests: mocked tools with deterministic evidence.
- Security tests: tenant isolation, secret leakage, prompt injection, SSRF and authorization bypass.
- End-to-end: connect → ask → investigate → approve → execute → verify.
- Build a synthetic startup-event dataset and measure groundedness, evidence coverage, tool selection, action correctness, false positives and execution success.

## 19. Reliability & Observability

- Timeout, retry and bounded backoff for external calls; circuit breakers for failing providers.
- Idempotency for writes and persisted checkpoints for agent runs.
- Structured logs and request/run/tool IDs.
- Track latency, token usage, provider errors, retries, approval rate, false positives, duplicate actions and cost.

## 20. Deployment

- Frontend: Vercel or equivalent. Backend/worker: Railway, Render or equivalent container host. PostgreSQL and Redis as managed services.
- Use HTTPS, production OAuth callback domains, managed secrets, database backups and tested restoration.
- Keep staging and production credentials separate.

## 21. Advanced Roadmap

- Add Jira/Linear, Calendar, CRM, Intercom/Zendesk, Stripe, product analytics and cloud monitoring.
- Add scheduled executive briefings, recurring workflows, organization-specific policies, department-specific agents and learning from approved/rejected actions.
- Add structured workspace memory: short-term, stable workspace facts and episodic investigation outcomes with retention controls.

## 22. Acceptance Scenario

- Connect test GitHub, Gmail and Slack.
- Seed a scenario where a release causes checkout complaints across email/Slack and matching code changes exist.
- Ask: What needs my attention today? Agent identifies the signal and evidence.
- Ask: Investigate it. Agent produces timeline, evidence, hypothesis and confidence.
- Ask: Prepare the fix workflow. Agent proposes GitHub issue + Slack notification + customer draft.

- Approve only the GitHub issue. Agent creates and verifies it; other actions remain unexecuted.
- Audit page displays the full chain.

## 23. MASTER COMMAND FOR ANTIGRAVITY

You are the principal software architect and implementation engineer for StartupOps AI.

The attached StartupOps\_AI\_Antigravity\_Complete\_Project\_Specification.pdf is the MASTER SOURCE OF TRUTH.

1. Inspect the entire repository before changing code.
2. Create PROJECT\_IMPLEMENTATION\_PLAN.md mapping every requirement to modules, files, database tables, APIs, UI, tests and milestones.
3. Reuse existing working code; do not delete working functionality without documenting why.
4. Implement the system in the exact phases in this document and keep it runnable after every phase.
5. Use Next.js/TypeScript/Tailwind, Python/FastAPI, PostgreSQL, Redis, LangGraph and Gemini via Google AI Studio.
6. Implement GitHub, Gmail and Slack as the first live integrations.
7. Use OAuth 2.0 and least-privilege scopes.
8. Keep all API keys and provider secrets server-side and out of source control.
9. Treat all retrieved external content as untrusted data and defend against prompt injection.
10. Enforce tenant isolation and typed tool authorization.
11. The LLM must never bypass policy or approval.
12. Every write action must be a structured action object with risk, target, parameters, approval state and idempotency key.
13. Risky actions require explicit human approval.
14. Verify every executed action and report partial failures honestly.
15. Add unit, integration, security and end-to-end tests.
16. Maintain ARCHITECTURE.md, SECURITY.md, API.md and DEPLOYMENT.md.
17. Do not substitute fake buttons for real integrations once an API is available.
18. Use mocks only in tests.
19. After every phase run tests, lint/type checks and build validation.
20. Start now by inspecting the repository and generating PROJECT\_IMPLEMENTATION\_PLAN.md, then implement Phase 1.

## 24. Definition of Done

- Workspace authentication and tenant isolation work.
- GitHub, Gmail and Slack can be connected and tested.
- Agent answers live questions using connected data.
- Cross-system investigations show evidence and hypotheses.
- Action plans are structured and reviewable.
- Write actions cannot execute without required approval.
- Execution is verified and persisted.
- Audit logs cover sensitive operations.
- Security and end-to-end tests pass.
- Local setup and production deployment are documented.

## 25. Deliverables

- Working frontend/backend; database migrations; OAuth integrations; typed tool registry; policy engine; LangGraph workflows; signal/correlation engine; approval/execution system; audit/observability; security tests; Docker setup; deployment configuration; documentation and runbooks.